Java 핵심 개념 정리

String 클래스 · 생성자 중복 제거 · this()와 this

이 자료의 핵심

문자열 리터럴은 String Pool을 활용하고, ==는 참조, equals()는 내용을 비교한다. String은 불변 객체이므로 반복적인 문자열 변경에는 StringBuilder가 일반적인 선택이다. 생성자 중복은 this()로 줄이고, this는 현재 객체를 가리킨다.

구분	핵심 질문	한 줄 답
String	문자열은 어떻게 저장·비교하는가?	참조형이며, 내용 비교는 equals()
불변성	문자열을 바꾸면 원본도 바뀌는가?	아니요. 보통 새 String 객체 생성
생성자	언제, 왜 호출되는가?	객체 생성 시 초기 상태 설정
this()	생성자 중복을 어떻게 줄이는가?	같은 클래스의 다른 생성자 호출
this	무엇을 가리키는가?	현재 생성된 객체 자신

먼저 바로잡을 점

String str = "ABC";가 내부적으로 항상 new String("ABC")와 똑같이 동작하는 것은 아니다. 리터럴은 String Pool의 객체를 재사용할 수 있지만, new String(...)은 일반적으로 별도의 객체를 만든다.

1. String 클래스 - 자바의 문자열 처리

1-1. String은 기본 자료형이 아니라 클래스

String은 참조 자료형이다. 변수에는 문자열 자체가 통째로 들어가는 것이 아니라, String 객체를 가리키는 참조값이 저장된다.

```
String str1 = new String("ABC");  // 명시적 객체 생성
String str2 = "ABC";              // 문자열 리터럴 사용
```

리터럴 문법은 간단하지만, new String(...)의 단순한 축약형이라고 이해하면 안 된다.

1-2. String Pool과 객체 재사용

같은 문자열 리터럴은 JVM의 String Pool에서 동일한 객체를 재사용할 수 있다.

```
String a = "ABC";
String b = "ABC";

System.out.println(a == b);      // 보통 true: 같은 Pool 객체 참조

String c = new String("ABC");
System.out.println(a == c);      // false: 서로 다른 객체
System.out.println(a.equals(c)); // true: 문자열 내용은 같음
```

시험 포인트

리터럴끼리 == 결과가 true로 나올 수 있어도, String의 내용 비교에는 항상 equals()를 사용하는 습관이 안전하다.

1-3. ==와 equals()의 차이

표현	비교 대상	String에서의 올바른 용도
str1 == str2	두 변수가 같은 객체를 참조하는지	객체 동일성 확인
str1.equals(str2)	문자열의 실제 내용이 같은지	문자열 값 비교

Null 안전 비교

str이 null일 가능성이 있으면 "ABC".equals(str)처럼 확실히 null이 아닌 문자열에서 equals()를 호출하면 NullPointerException을 피할 수 있다.

1-4. String의 불변성(Immutable)

String 객체는 한 번 생성되면 그 내부 문자열 값을 바꿀 수 없다. 연결 연산 결과는 기존 객체를 수정하는 것이 아니라 새로운 문자열을 만든다.

```
String str = "ABC";
str = str + "D";  // "ABCD" 결과를 가진 새 String을 str이 참조
str = str + "E";  // "ABCDE" 결과를 가진 새 String을 str이 참조
```

개념적으로는 중간 문자열인 "ABC", "ABCD", "ABCDE"가 생길 수 있다. 더 이상 참조되지 않는 객체는 가비지 컬렉션(GC)의 대상이 된다.

정확한 표현

반복 연결이 많은 코드는 임시 객체와 복사 비용을 늘려 성능과 메모리 사용에 부담을 줄 수 있다. 하지만 이것을 곧바로 메모리 릭(memory leak)이라고 부르는 것은 부정확하다. 참조가 사라진 객체는 보통 GC가 회수할 수 있다.

1-5. 자주 바뀌는 문자열: StringBuilder / StringBuffer

클래스	특징	주 사용 상황
String	불변	문자열 변경이 적고 읽기 위주
StringBuilder	가변, 동기화 없음, 일반적으로 더 빠름	단일 스레드에서 반복 연결 - 보통 우선 선택
StringBuffer	가변, 메서드 동기화 지원	여러 스레드가 동일 객체를 공유해야 하는 경우

```
StringBuilder sb = new StringBuilder("ABC");
sb.append("D");
sb.append("E");
String result = sb.toString();  // "ABCDE"
```

단, 간단한 한두 번의 + 연결까지 무조건 StringBuilder로 바꿀 필요는 없다. 컴파일러가 일부 연결을 최적화하기도 한다.

2. 생성자와 중복 코드 제거

2-1. 생성자의 문법과 역할

- 메서드와 비슷하지만 리턴형을 작성하지 않는다. void도 붙이지 않는다.
- 이름은 반드시 클래스명과 같아야 한다.
- new로 객체가 생성될 때 호출되어 멤버변수의 초기 상태를 설정한다.
- 한 클래스 안에서 파라미터 목록을 다르게 하여 생성자 오버로딩이 가능하다.

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

Student s = new Student("김학생", 20);
```

2-2. 생성자 오버로딩

생성자 이름은 모두 클래스명으로 같으므로, 자바는 파라미터의 개수·자료형·순서를 보고 어떤 생성자인지 구분한다. 파라미터 이름만 바꾸는 것은 오버로딩이 아니다.

가능 여부	예	이유
가능	Student() / Student(String name)	파라미터 개수가 다름
가능	Student(String name) / Student(int age)	파라미터 자료형이 다름
가능	Student(String n, int a) / Student(int a, String n)	파라미터 순서가 다름
불가능	Student(String name) / Student(String nickname)	파라미터 이름만 다름

2-3. 오버로딩에서 생기는 중복

각 생성자에서 동일한 필드 초기화 코드를 반복하면 수정 지점이 늘고 실수 가능성도 커진다.

```
class Student {
    String name;
    int age;
    String major;

    Student(String name) {
        this.name = name;
        this.age = 20;
        this.major = "미정";
    }

    Student(String name, int age) {
        this.name = name;       // 중복
        this.age = age;
        this.major = "미정";   // 중복
    }
}
```

이때 같은 클래스의 다른 생성자에 초기화를 맡기는 문법이 this()이다.

2-4. 기본 생성자 주의

> **기본 생성자는 언제 자동 제공될까?**
>
> 클래스에 생성자를 하나도 작성하지 않았을 때만 컴파일러가 파라미터 없는 기본 생성자를 자동으로 추가한다. 생성자를 하나라도 직접 작성하면 Student() 같은 무인자 생성자는 자동으로 생기지 않는다.

3. this()와 this

3-1. this() - 현재 클래스의 다른 생성자 호출

this(...)는 같은 클래스 안의 다른 생성자를 호출하여 초기화 코드를 한곳으로 모으는 생성자 위임 문법이다.

```
class Student {
    String name;
    int age;
    String major;

    Student() {
        this("이름 없음", 20, "미정");
    }

    Student(String name) {
        this(name, 20, "미정");
    }

    Student(String name, int age) {
        this(name, age, "미정");
    }

    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
        this.major = major;
    }
}
```

가장 많은 정보를 받는 생성자에 실제 초기화 코드를 모으면 중복을 줄이고 기본값도 한눈에 관리할 수 있다.

3-2. this() 사용 규칙

규칙	설명
생성자 안에서만 사용	일반 메서드에서는 this()로 생성자를 호출할 수 없음
반드시 첫 번째 문장	this()보다 앞에 다른 실행문을 둘 수 없음
한 생성자에서 한 번	첫 문장이어야 하므로 여러 번 호출할 수 없음
순환 호출 금지	A 생성자 → B 생성자 → A 생성자처럼 되돌아오면 컴파일 오류
super()와 동시 사용 불가	둘 다 생성자의 첫 문장이어야 하므로 직접 함께 쓸 수 없음

```
Student() {
    System.out.println("start");
    this("홍길동");  // 컴파일 오류: this()는 첫 문장이어야 함
}
```

3-3. this - 현재 객체 자신

괄호 없는 this는 현재 실행 중인 인스턴스 객체 자신을 가리키는 참조다.

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
        // this.name : 현재 객체의 멤버변수
        // name      : 생성자의 매개변수
    }

    void printName() {
        System.out.println(this.name);
    }

    Student getThis() {
        return this;
    }
}
```

멤버변수와 매개변수 이름이 같을 때 this.멤버변수로 구분하는 용도가 가장 흔하다.

3-4. this()와 this 비교

구분	this()	this
의미	현재 클래스의 다른 생성자 호출	현재 객체 자신을 가리키는 참조
형태	this(인수);	this.필드, this.메서드(), return this
사용 위치	생성자 내부의 첫 번째 문장	인스턴스 생성자·인스턴스 메서드
주요 목적	생성자 코드 중복 제거	멤버 접근, 이름 충돌 해결, 현재 객체 전달
static 영역	사용 불가	사용 불가 - 특정 객체가 없기 때문

4. 최종 암기 정리

번호	반드시 기억할 문장
1	String은 클래스이며 변수에는 객체의 참조값이 저장된다.
2	String 내용 비교는 ==가 아니라 equals()를 사용한다.
3	문자열 리터럴은 String Pool의 객체를 재사용할 수 있다.
4	String은 불변이므로 변경 결과는 보통 새 문자열 객체다.
5	반복 연결은 보통 StringBuilder, 동기화가 필요하면 StringBuffer를 고려한다.
6	생성자는 리턴형이 없고 이름은 클래스명과 같다.
7	생성자 오버로딩은 파라미터 목록이 달라야 한다.
8	this()는 같은 클래스의 다른 생성자를 호출하며 반드시 첫 문장이다.
9	this는 현재 객체 자신을 가리킨다.
10	this와 this()는 static 문맥에서 사용할 수 없다.

한 문장으로 연결하기

String의 내용은 equals()로 비교하고, 자주 이어 붙일 때는 StringBuilder를 쓴다. 객체의 초기화는 생성자가 담당하며, 생성자끼리 중복되면 this()로 위임하고, 현재 객체의 멤버를 가리킬 때는 this를 쓴다.

빠른 확인 문제

문제	정답
new String("A") 두 개의 내용을 비교하려면?	equals()
생성자가 하나도 없을 때 자동 제공되는 것은?	파라미터 없는 기본 생성자
다른 생성자를 부르는 문법은?	this(...)
this()는 생성자 안의 몇 번째 문장이어야 하는가?	첫 번째 문장
현재 객체의 필드 name을 명확히 쓰면?	this.name